EEET2588

Real-time Systems Engineering & Design

Lecture 6: Task Structuring Criteria
- Task Minimization

Dr Linh Tran

Course note acknowledgment: *Original course notes developed by Roy Ferguson and Samuel Ippolito from RMIT Melbourne*

Task Structuring Criteria (Gomaa)

- Large systems will generally have lots of objects with complex relationships
  - Effort needs to be put into identifying how the system should be arranged
    - 1 task per I/O device
    - 1 task per state machine
    - 1 task per system independent

    } Leads to a lot of tasks which need to be organised within the correct structure of processes and threads.
  - Iterative design approaches can be employed (this may require re-working parts)
  - It would be better to apply a formalized approach
    - Using a Task Structuring Criteria as discussed in lecture 5
      - 11 point Partitioning the Solution into Processes and Threads from Structured Analysis and Design (SA/SD)
      - Concurrent Design Approach for Real-time Systems (CODARTS)
      - Gomaa's UML version named: Concurrent Object Modeling and Architectural Design (COMET)
  - There are several other major Task Structuring methodologies that have been proposed. See Gomaa, Williams, etc.
- For real time systems, there are 2 main types of task structuring criteria:
  - Periodically
  - Aperiodically
  - What about continuous?

Task Structuring Criteria (Gomaa)

- Sometimes it is not obvious how tasks should be grouped
- Gomaa proposed a further method called Task Structuring Criteria
  - Breaks down into five main categories to group tasks:
    1. I/O task structuring criteria
    2. Internal task structuring criteria
       - Control task, state machine, etc.
    3. Task priority criteria
       - Very important in a system such as QNX with real time scheduling
    4. Task clustering criteria (to reducing the number of tasks)
    5. Task inversion criteria
  - *see Gomaa, Chapter 14*
- The first 3 categories are used to separate the tasks out
- The last 2 categories are used to group the tasks to avoid unnecessary task switching and minimize the number of threads/processes in the system
  - Maintenance of system
  - Minimize excessive task switching

1. I/O Task Structuring Criteria

- **Depends on I/O hardware characteristics**
  - Allocate a task for every I/O device (simplest form)
    - **Asynchronous** (interrupt driven) – Typically no choice but to have one task per device
    - **Passive**
      - polled inputs or periodic output
        - Task runs periodically to check the input or on demand (i.e. State machine driven)
        - Potential to group I/O tasks if timing matches
    - **Communications link** (protocol related e.g. TCP/IP)
    - **Periodic** I/O tasks
      - External timer driven - frequency jitter is an issue
      - ADC (analogue digital converter) has its own internal timer
- **Also depends on nature of data**
  - Discrete (boolean or finite number of values)
  - Continuous (e.g. analog)

2. Internal Task Structuring Criteria

- **4 Main types of tasks:**
  - **Periodic internal tasks**
    - Internal timer in the CPU which activates to perform periodic functions
    - QNX periodic timer (periodic signal pulse, thread, message receive, etc.)
  - **Asynchronous internal tasks**
    - Activated by internal events or messages/pulses
  - **Control task**
    - Executes a state chart (or more likely a state machine)
  - **User Role task**
    - Handle user I/O (keyboard and display)
      - Similar to `scanf()` or `gets()`
      - Terminal input
        - Multiple terminal windows will probably need separate tasks to handle input
    - May be one task per terminal/display or window

3. Task Priority Criteria

- **Task priority is extremely important in RT systems**
  - QNX provided 255 different levels of priority
- **Time Critical Tasks are usually set to high priority and often need to be separate tasks**
  - Hard deadlines
  - High priority
  - May be structured as separate tasks
  - Not safe to combine with less critical tasks
- **Non-time critical and computationally intensive tasks should have a low priority**
  - Tasks should not dominate CPU time – starvation of critical tasks needs to be avoided
  - Normally RT Systems don't have computationally heavy tasks, but if it does:
    - Dedicated hardware can be used to deal with it, e.g:
      - 3D Image rendering or data processing
      - MP3 or DiVx processing ICs
    - Else must run as low priority and be preemptable as a lossy system

4. Task Clustering Criteria

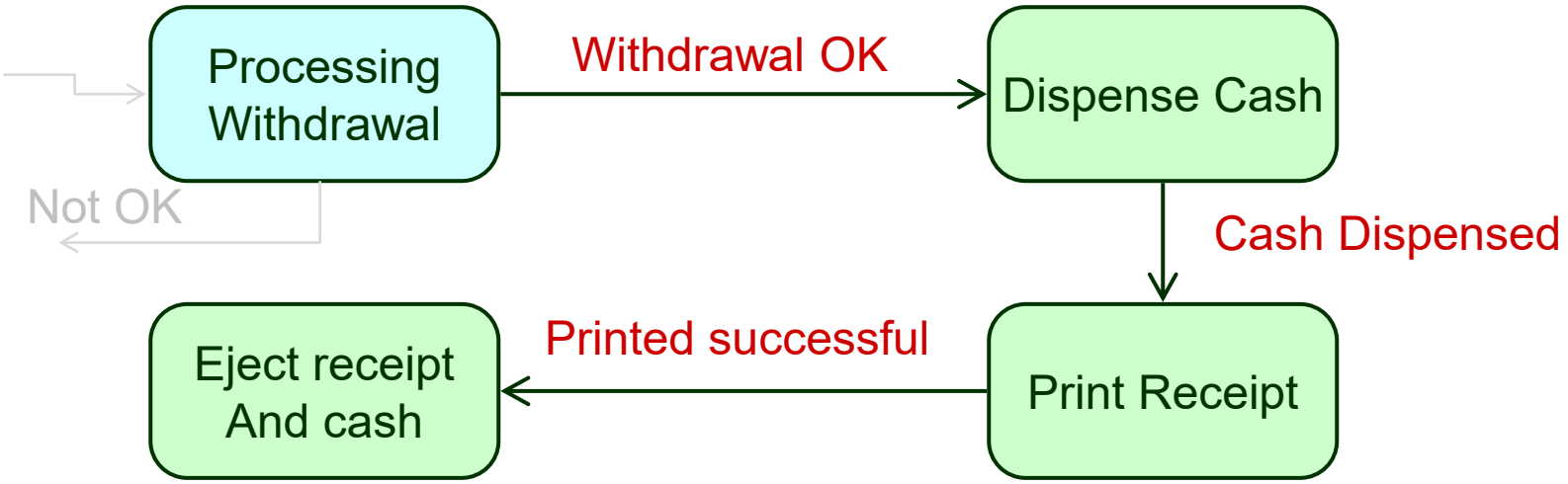
- **During the Analysis model phase it can appear as many of the objects are potentially running concurrently.**
  - Could lead to a **large number of small tasks** which would increase system complexity and execution overhead
  - **Excessive task switching** leads to high execution over head (System overhead)
    - Time slicing of `4 msec` is very long on the scheme of things, however if you make it smaller the system overhead is increased.
- **Clustering Tasks (unrelated) together will reduce system overhead**
  - Task clustering criteria can be used to determine which tasks can be **combined** into a single thread/process without changing the behavior of the system
  - Needs to be well document as it may not be obvious why certain operations are paired together. (ie. Common timing requirements, I/O related, etc..)
  - An experienced designer may perform task clustering during task identification
    - Suitable concurrent objects can be identified early on
    - Less experienced designers may need to use a system profiler to identify excessive task switching or try multiple task clustering arrangements to optimize performance.

4. Task Clustering Criteria - Temporal Clustering

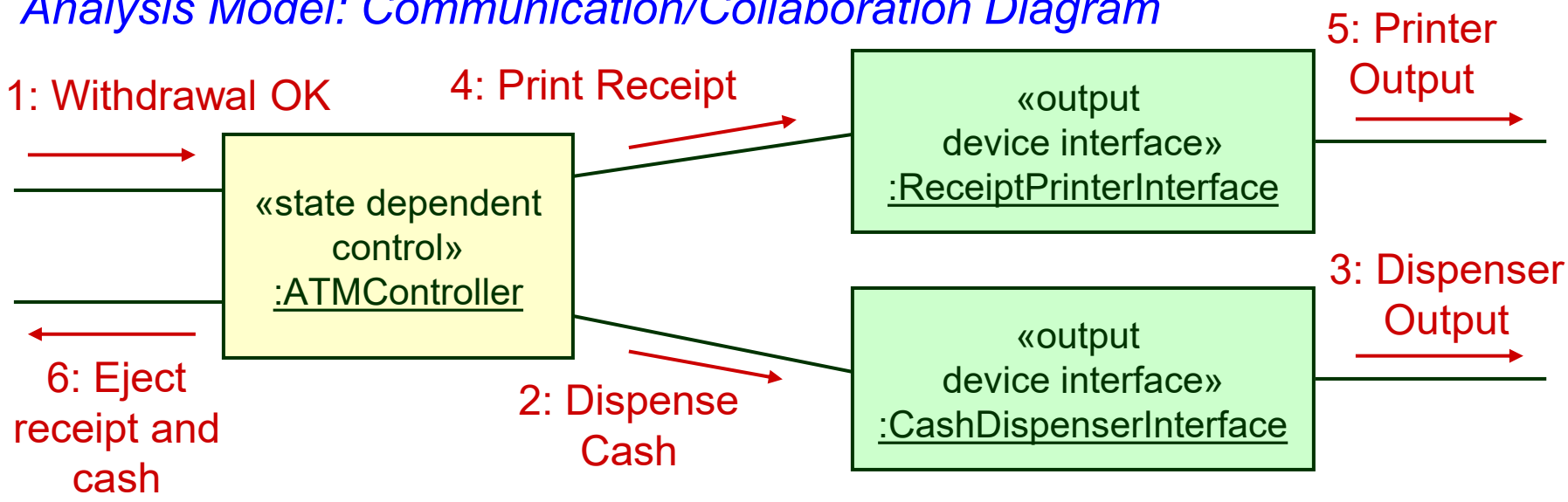
- **Temporal clustering**
  - May combine tasks which are activated by the same event (often periodic)
    - But hard to combine if periods are different
    - May try to combine tasks with related periods
  - Give preference to clustering tasks that are functionally related and likely to be of equal importance in scheduling
  - May not combine time critical tasks with less time critical tasks
- **Sequential Clustering**
  - Operations that must occur in a sequential order may be combined into one task
- **Control Clustering**
  - A control task may be grouped with data transformations it activates
- **Mutually Exclusive Clustering**
  - May combine functions that can never execute concurrently
  - May be triggered by different events
- **When not to combine:**
  - Don't combine if differing priorities
  - Question if it will help to combine tasks if they can execute on separate CPU cores (or nodes) concurrently

4. Task Clustering Criteria - *Analysis Models*

Analysis Model: State Chart of the last part of an ATM machine



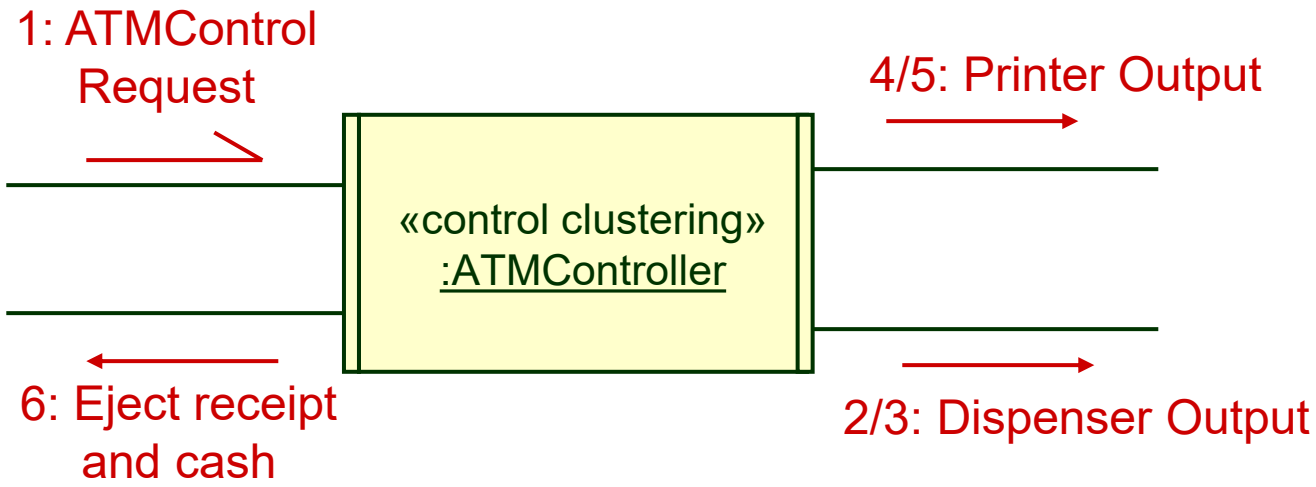
Analysis Model: Communication/Collaboration Diagram



4. Task Clustering Criteria – Task Minimization

- Given that the ReceiptPrinterInterface and CashDispenserInterface are controlled by the ATMController task, then it might be possible to combined them all into one overall task.
- Thus in the final Design Model, the separate task shown in the Analysis model can be reduced to:

Final Design Model: Collaboration/Communication Diagram



- Solution depends on the fact that the ATMController task is the only task that controls the case dispenser and printer output.

5. Task Inversion

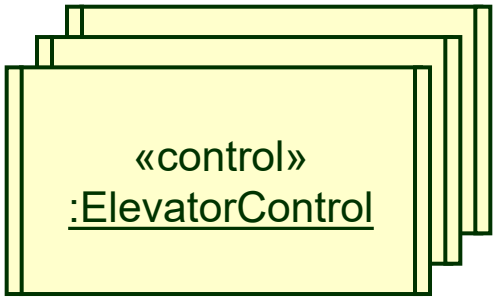
- Similar to task clustering, task inversion is used to reduce tasking overhead
- **Task inversion is another method to reduce the number of tasks**
  - Reducing the number of tasks in a system in a systematic manner
  - Extreme case - Map concurrent solution to a sequential solution
    - Or reduce all tasks to 1 task

- **Three types of Task Inversion:**
  1. Multiple instance
     - Group tasks that have the same functionality
  2. Sequential tasks
     - Group tasks that use messages to synchronize sequential events
  3. Temporal tasks
     - Similar to temporal clustering, where periodic tasks can be grouped

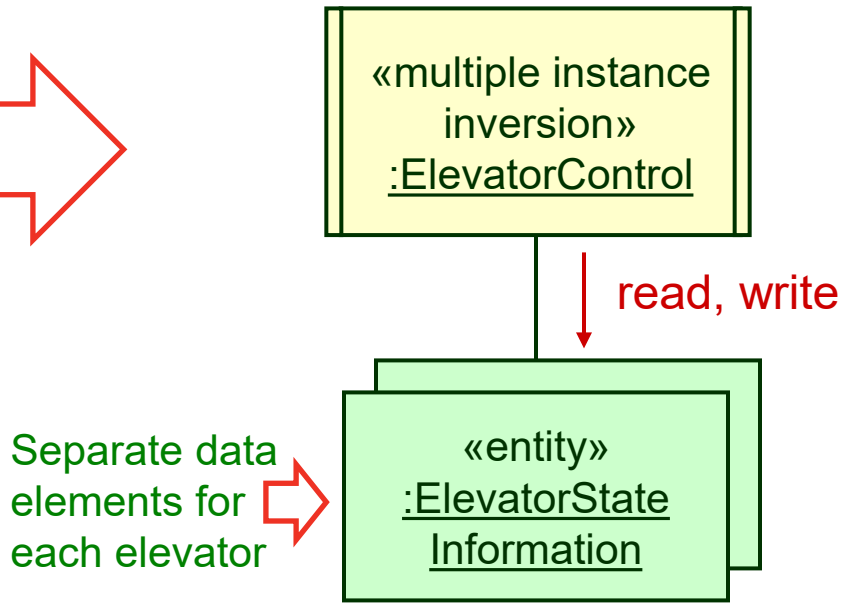
5. Task Inversion - Multiple Instance (same functionality)

- Replace "identical" tasks of the same type by one task that performs the same service.
  - Each object's state information is captured in a separate passive entity object.

Design Model:
One Task for each Elevator



Design Model – Task Inversion:
One Task for all Elevators



- This makes sense if you only have 1 CPU
  - However if running on a multi-core CPU then it might make sense to keep them as separate tasks.
  - A single CPU would have extra overhead to maintain the separate states of each elevator.

5. Task Inversion - Sequential Inversion

- **If there is tightly coupled communication between two or more tasks, these tasks can be assessed for their suitability for reduction by Sequential Task Inversion.**
- For instance, tasks that use Message passing to communicate to each other can be combined.
  - Replace message passing between tasks to simple function calls.
  - Only to be used if the behaviour of the system is unaffected by the grouping.
- In the case of a 'produce' and consumer, instead of sending messages between the produce to the consumer, rather, combine the tasks together to avoid the messaging.
  - Inversion of the consumer with respect to the producer operations may occur
  - If combined the message data becomes call parameters

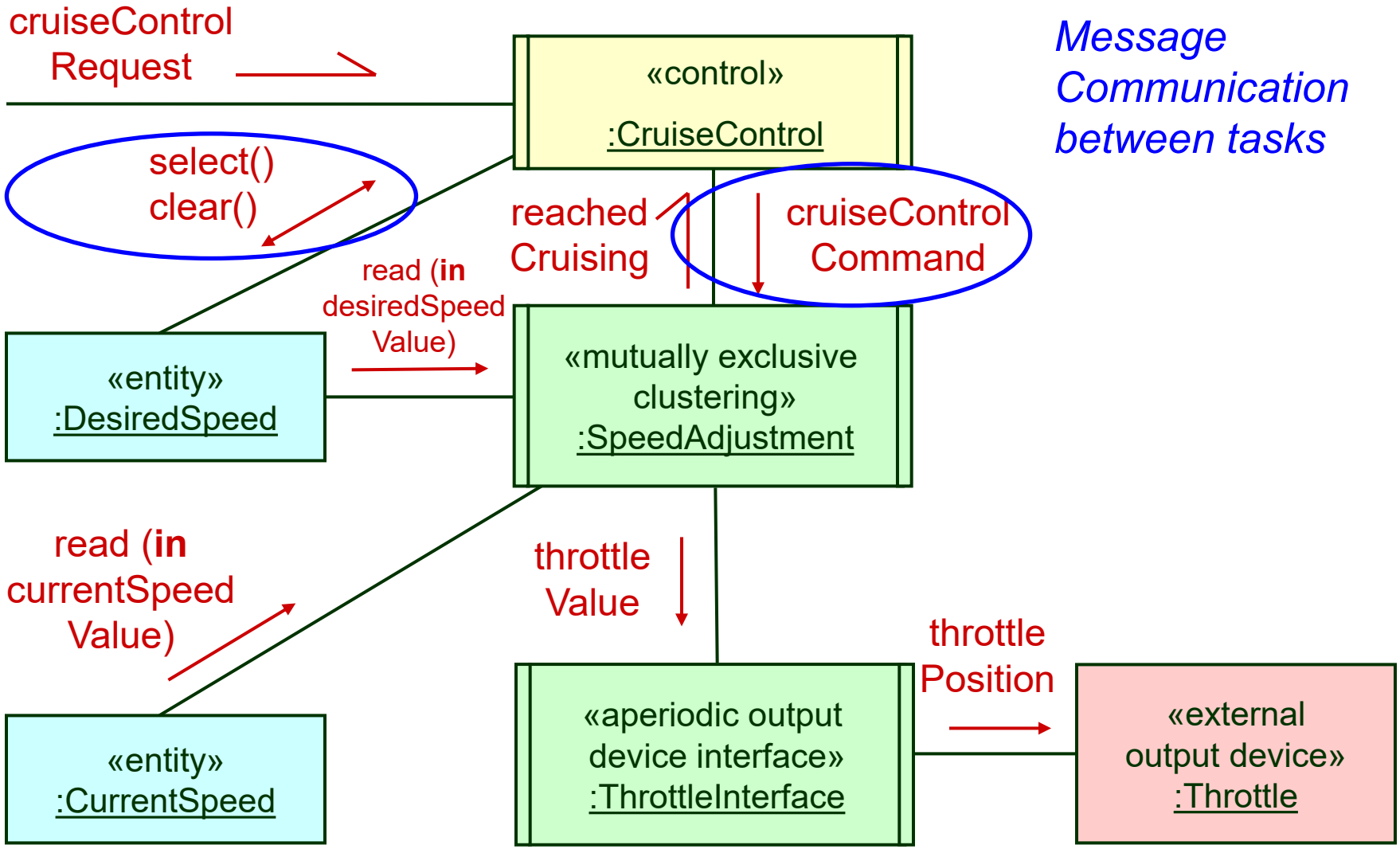
    Producer function produces the data and then calls:

    ```
    consume_data( &data_message );
    ```

    thus reducing the need for a process switch

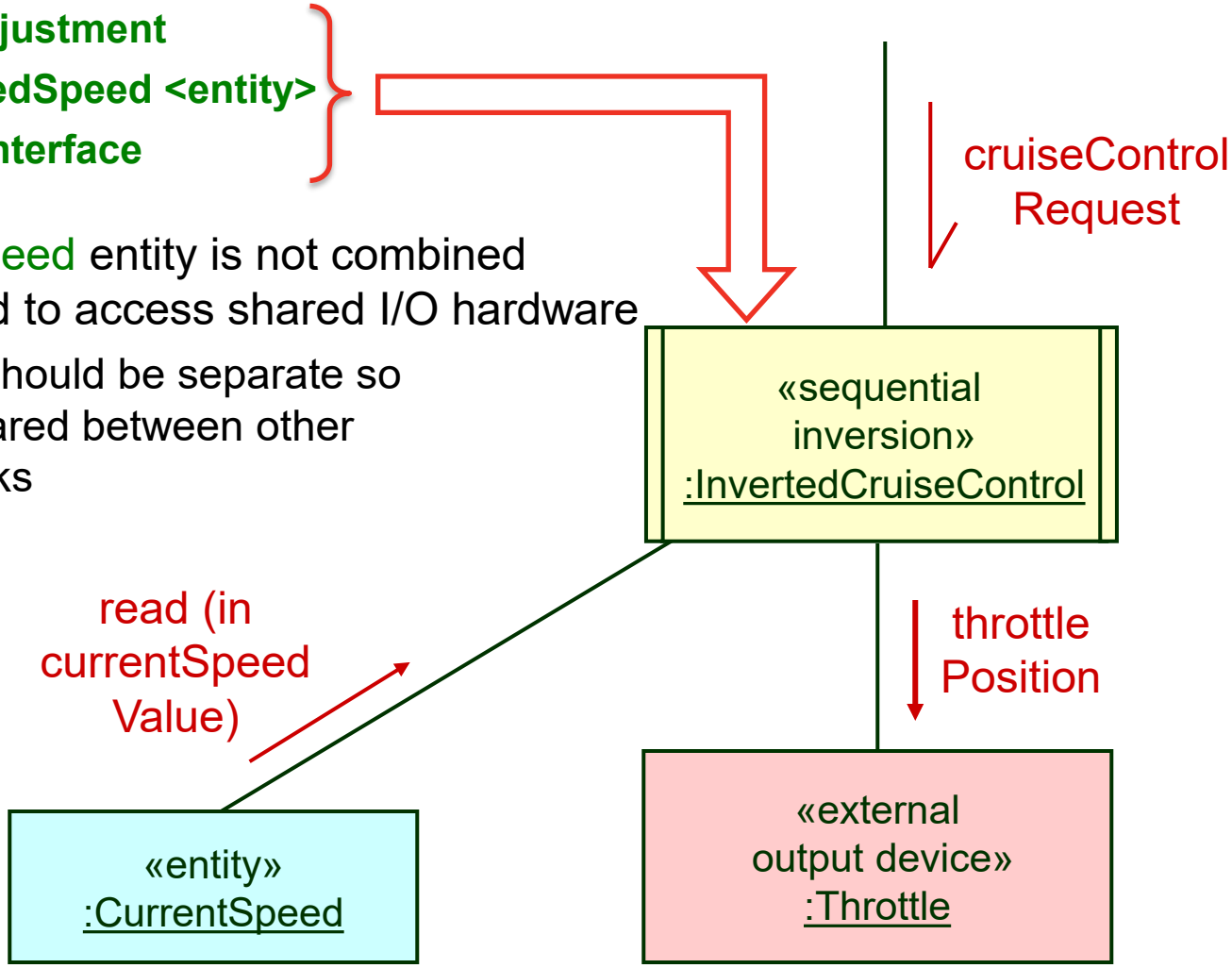
5. Task Inversion - Sequential Inversion

- Sequential Task Inversion Example
  - Cruise Control with separate tasks



5. Task Inversion - Sequential Inversion

- Sequential Task Inversion Example which uses function calls
  - Enables us to combine **CruiseControl** (state machine process) with:
    - **SpeedAdjustment**
      - **DesiredSpeed <entity>**
    - **ThrottleInterface**
- The CurrentSpeed entity is not combined as it is required to access shared I/O hardware
  - I/O access should be separate so it can be shared between other systems/tasks

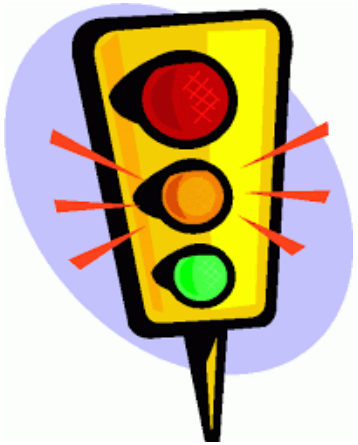


5. Task Inversion - Temporal Inversion

- **Two or more**
  - **periodic internal timers**
  - **periodic I/O, and/or**
  - **periodic temporally clustered tasks**

  can be combined
- All task periods need to be related
  - Similar to the weak forms of temporal clustering, which states that you
    - may combine tasks which are activated by the same event (often periodic)
  - Task are typically timer driven
    - On each activation the task decides which function (of the original separate task functions) is to be performed
- Not good from a design viewpoint
  - But maybe a necessary optimisation to reduce tasking overhead as creating multiple timing resources is wasteful if there is a timing relationship been events.

Group Project Advice



- For many of you this will be your first look at a highly concurrent multi node system
- Looking for High Marks? – well attempt some of these:
  - Based on a real intersection
  - Implemented trains from both directions
  - Multiple controllers (smart controllers, update patterns, etc.)
  - Test Plan
  - Use custom hardware or GUI based interface…
  - Fault tolerance - failure between nodes handled (including recovery mechanisms)
- Initial Design Report
  - Flesh out basic design
  - Define project boundaries (ie. Size of project, features that it will/won't support)
  - Create a reference to refer too to keep you on track
- Final Report
  - Should bridge the logic gaps over the initial design report

Workshopping the problem…

- Three main ingredients behind developing a solution to a problem:
  - Fact based
  - Hypothesis driven
  - Structural
- The design work comes within the structure.
  - It is not so much what the actual structure is, rather that the problem has (or is given) a structure and the structure is logical and coherent. (we can optimize later)
- Ideally, the structure should drive the problem-solving process.
  For example:
  - What is the problem?
  - How do we scope it?
  - How do we break it out into its constitutes elements?
  - How do we go about getting the required input/output data (external data, internal communication channels, etc.)
  - How do we then present it to the developers/clients in a way that they can implement/understand the design.

Relating the process to the real world …

- **The Project is simulating a small industrial project/tender process**
- **The diagrams should be able to allow the client, designers and implements to draw conclusions regarding the proposed solutions.**
  - Gauge size of project, Scope of project, expected man hours, cost, etc.
- **The diagrams should provide a mechanism to move up and down:**
  - The hierarchy of the problem
  - The hierarchy of the proposed design solution
  - Identify potential problems within the design (at various levels within the hierarchy)
  - Reflect on the levels of thinking behind the design.
- **In the real world, consultant might be engaged to:**
  - Define the problem or requirements
  - Propose a solution to the problem (i.e. give it structure)
  - Implement the design
  - Provided a structured testing plan
  - Document any of the above stages, or
  - Backfill a report to justify any of the above stages (after a decision has already been made)
- Good Luck… but plan ahead and start early…

Acknowledgment

I would like to Thank and Acknowledge the support of: Dr. *Samuel Ippolito*, the Course Coordinator of corresponding “EEET2166 /1262 Real-time Systems Engineering & Design” course from RMIT Melbourne.